The Alexandria Problem

The Library of Alexandria was the ancient world's greatest repository of knowledge, and its greatest single point of failure. It didn't burn in a single night. It declined across decades, through neglect, through the slow erosion of what it had been. By the time it was gone, almost no one had noticed it going.

If the understanding holds today, how does it survive the people who leave?

I learned to be an engineer in code reviews, and in the pairing sessions and planning meetings around them.

From watching seniors think out loud, never from documentation or courses: how they searched for the problem beneath the problem, how they argued about tradeoffs, how they asked questions that reframed everything before anyone had looked at a single line of code.[1]

The most important thing I learned was how to ask the right question. How to formulate a problem, and the motivation behind it, so clearly that the solution could emerge from it, not knowing the answer, but knowing how to construct the context that makes the answer findable.

I learned that most deeply in open source. Suddenly the person reading the issue knows nothing. They weren't in the meeting. They don't know the system, the constraints, the previous decisions. I have to build the entire picture from scratch: structure the problem so a stranger can understand it, provide every piece of context someone outside my head would need to engage with it meaningfully.

That's a harder discipline than explaining something to a colleague who shares the same background. A colleague fills in the gaps from shared experience. A stranger doesn't. Open source teaches the contributor to make the problem legible to someone with no obligation to

[1]This is Peter Naur's claim from "Programming as Theory Building" — cited, in full, in the introduction. Naur held that a new programmer can only acquire the theory of a program through active engagement with the people who already hold it, and that the theory itself cannot be written down. He drew the conclusion at maximum strength: a program whose team has left should be discarded and rewritten, because the theory died with the room.

figure out what they meant.

But there's a second discipline open source teaches that's equally important. Not over-sharing.

As little as possible, as much as needed. Too little context and the stranger can't engage. Too much and the contributor has made their problem the stranger's problem. They have to sort through noise to find what actually matters. The discipline is knowing what to leave out: what's assumed, what's load-bearing, what would change the answer if it were different.

It's the same discipline as writing a good prompt. Not the longest prompt, the tightest one. Too much context and the agent loses focus, chases the noise, produces something that addresses everything it was told and misses what was meant. Too little and it fills the gaps with confident guesses. The discipline runs past the single prompt: knowing when to clear a session that has run too long and start fresh, scoped to the real problem, is part of the same skill.

There's a failure mode minimum viable context is designed to prevent. The XY problem.[2] Someone has problem X, thinks the solution is Y, asks for help with Y, and the person helping them never finds out about X. The answer to Y lands, it doesn't solve anything, and everyone wasted time.

It's endemic in open source. Someone asks a very specific question about a very specific implementation detail, and three comments in a maintainer asks "what are you actually trying to do," and the real problem surfaces. Which has a completely different, usually simpler solution.

It's equally endemic in AI prompting. The engineer prompts for Y, the agent implements Y confidently, and X remains unsolved. The agent doesn't ask what they're actually trying to do. It assumes Y is the right question because they asked it.

Minimum viable context means surfacing X before asking about Y. It means providing the right context, not just enough of it. The actual problem, not the assumed solution.

That's what the senior in the code review was doing with "what are you actually trying to solve here": checking for the XY problem before engaging with the implementation.

When I watch junior engineers struggle with AI tools today, it's almost never a technical problem.

It's a problem formulation problem.

They're bad at structuring the question, not because they're bad engineers but because they haven't yet developed the habit. They go straight to the agent with a vague sense of what they want. The agent produces something. They take the output and keep going.

[2]Coined by Mark Jason Dominus in a 2001 comp.lang.perl.misc post; popularized since via xyproblem.info.

The senior in a code review would have stopped them before that. "What are you actually trying to solve here?" Asked enough times, by enough seniors, in enough reviews, that question is how they internalize it. How they start asking it unprompted before anyone else has to.

Without that environment, the junior keeps getting output. Just not always the right output. And they don't yet have the experience to tell the difference between a good answer to a bad question and a good answer to the right one.

The AI doesn't teach that distinction. It's too helpful. It tries regardless of how vague the input is. It doesn't say "your question is too vague." That's useful once they know how to evaluate the output. It's a trap when they don't yet know what good looks like.

Mob programming works because everyone is in the thinking together. The navigator holds the direction. The driver holds the keyboard. Everyone else watches, questions, catches what the driver missed, builds the shared understanding that makes the session worth more than the code it produces. The output is almost secondary. The point is what happens in the room while it gets made.

Then someone, not the navigator, just someone, started prompting on their own machine. Through efficiency, not malice. The agent was faster. A solution arrived. The group looked at it. We moved on.

The mob didn't break dramatically. Nobody announced a change of approach. Someone just quietly did it differently, the output appeared, and the group followed the path of least resistance. The session continued. The mob stopped. The fun went with it, and so did the learning. Knowledge that used to build through watching each other reason, through the disagreement and the question and the "wait, what about this edge case," stopped accumulating the moment the output arrived pre-formed.

We were still in the room together. We had stopped thinking together.

That's the Alexandria Problem: not an event but a process. The library doesn't burn in one dramatic fire. It burns one session at a time, one private prompt at a time, one skipped conversation at a time. The knowledge that used to accumulate through proximity and repetition stops accumulating the moment the friction disappears.

And AI removes the friction so efficiently that no one notices the burning until the smoke is already gone.

It's the Codex problem from the first chapter, seen from the inside this time. What never leaves someone's head is exactly what the next agent can't reach, and a private prompt is how it stays

there.

It happened in code reviews, in pair programming, in the hallway conversation where a senior explained why the approach was wrong while the work was still an idea.

AI tools, used individually, break that transfer. Just by being available. The junior doesn't need to ask the senior. The agent is faster and less intimidating. The senior doesn't need to explain; they can just do it themselves.

The moment of friction that used to produce a teaching interaction gets resolved by the tool before it becomes a conversation.[3]

The junior develops their own AI workflow in isolation, without the senior's eye on the process — and what they're not developing is the underlying skill: the problem formulation, the question that focuses the work before a prompt is written.

The Spec Session is the new learning environment, arrived at by consequence rather than design.

When the team is in the room together, building the spec before the agent runs, the junior watches how seniors frame problems. They see the questions that get asked before anyone proposes a solution. They see an experienced engineer push back on a direction not because the implementation is wrong but because the problem statement isn't tight enough. They see what "ready for the agent" actually looks like, and more importantly, what it doesn't look like yet. They learn it from watching humans think through what needs to be generated and why, not from reading generated code.

The mob exploration works the same way. A new library, a new approach, a PoC the team runs together. The junior isn't just evaluating the technology. They're watching how experienced engineers evaluate tradeoffs. What questions they ask. What concerns they raise. What "good enough for now" means versus "this will hurt us later."

It runs both ways. We were evaluating high-availability tooling once: paid caching, Redis, the works. A junior asked why we didn't just poll and persist to a volume on the cluster. We'd been too deep in the sophisticated option to see the plain one. He dragged us out. He was learning how the team weighed tradeoffs by sitting in the room, and from outside the hole we'd dug, he asked the question we'd stopped asking.

[3]Anthropic put numbers on this in a randomized controlled trial: junior engineers learning a new library with AI assistance scored 17% lower on comprehension minutes later — nearly two letter grades — with the largest gap in debugging, while the speed gain was not statistically significant. How they used the tool decided the outcome: those who delegated generation or leaned on AI to debug scored lowest; those who asked conceptual questions or interrogated the generated code preserved their learning, and the conceptual-inquiry group, errors and all, was among the fastest. The authors note the study used a chat assistant, not an agentic tool, and expect agentic impacts to be more pronounced. Judy Hanwen Shen and Alex Tamkin, "How AI Assistance Impacts the Formation of Coding Skills" (Anthropic, January 2026; arXiv:2601.20245).

You can't get that from a spec document or from reading the agent's output. None of this is formal teaching; nobody stands at a whiteboard. Experienced engineers make their thinking visible, and the junior learns how the answer gets found.

If the team stops being the place where that transfer happens, everyone heads down with their own tools, and the next generation develops faster in some ways and not at all in the one that matters most.

It's not a prompt engineering course but a career spent in rooms where people think out loud, ask hard questions, and show the junior what careful problem formulation looks like by doing it in front of them.

Seniors develop prompting habits nobody else sees. Architects discover patterns nobody else inherits. Knowledge that used to become team knowledge remains personal knowledge, and there's no code review to surface it, no pairing session to transfer it, no hallway conversation to pass it on.

The junior feels the impact first, but the loss belongs to the whole team.

When the senior's attention shifts upward, from implementation to architecture, the junior is often left downstream, catching the behavioral edge cases the senior no longer has the bandwidth to notice. That division of attention can work, but only if it's deliberate.

The senior whose attention has shifted to the architectural plane has a new responsibility in the Spec Session: to pull the junior up with them. To ask the questions that make architectural thinking visible: why this approach, what happens at scale, what would break this in six months. And to ask them not just in private, but in the room, with the junior present and expected to engage — otherwise the junior develops a different and narrower skill: valuable, but not the same as learning to think architecturally.

The skill that makes a great engineer is the one thing the agent can't teach, because it's learned by watching a human do it. If the room goes quiet, the next generation inherits the tools without the judgment that makes the tools worth having.

But if the room is where that judgment transfers, how does anyone know the transfer is working? How does anyone know the understanding is actually shared and not just assumed?